

WHAT NEXT VISION MOTORS – Salesforce-Based Vehicle Management System

ABSTRACT

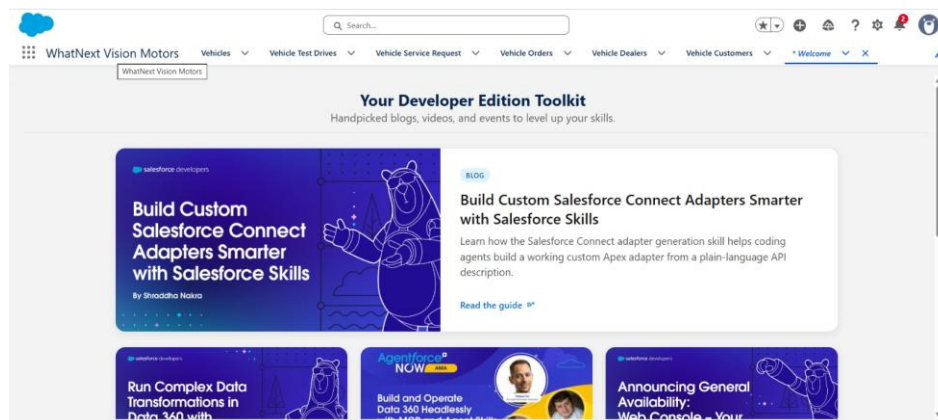
WhatNext Vision Motors is a Salesforce-based vehicle management system developed to centralize and automate important automotive business operations. The solution manages vehicle information, dealer information, customer records, vehicle orders, test drives, and service requests through a Salesforce Lightning application. The project combines declarative Salesforce configuration with programmatic automation using Record-Triggered Flows, Apex Triggers, a Trigger Handler, Batch Apex, Scheduled Apex, and automated email actions.

The main business requirements are to maintain vehicle and stock information, manage customer orders, prevent orders when a vehicle is out of stock, automatically assign a dealer using customer location, send scheduled reminders for test drives, and process pending orders when vehicle stock becomes available. The implementation demonstrates how Salesforce can be used to reduce repetitive manual work and improve the consistency of vehicle order processing.

1. WhatNext Vision Motors Salesforce Application

1.1 Application Overview

The WhatNext Vision Motors Lightning application provides a single workspace for the project's major business objects. The navigation includes Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Test Drive, Service Request, Reports, and Dashboard. The application brings the configured data model and automation features together so that the business process can be demonstrated from a single Salesforce app.



2. Project Objectives and Business Requirements

2.1 Project Objective

The objective of the project is to implement a Salesforce CRM solution for WhatNext Vision Motors that supports vehicle sales operations and customer interaction. The solution is designed around centralized data management and automation. Instead of relying on separate manual activities, the Salesforce application connects customer, vehicle, dealer, order, and test-drive information.

2.2 Key Business Requirements

- Store and manage vehicle details and stock availability in Salesforce.
- Store dealer information and maintain vehicle-to-dealer relationships.
- Store customer information required for vehicle orders and test drives.
- Track vehicle orders and their processing status.
- Prevent an order when the selected vehicle has no available stock.
- Reduce vehicle stock when a confirmed order is processed.
- Automatically assign a dealer by matching the customer's location with dealer location.
- Send an automated email reminder before a scheduled test drive.
- Process pending vehicle orders when new stock becomes available.
- Use Scheduled Apex to run the pending-order batch automatically.

2.3 Salesforce Technologies Used

The implementation uses Salesforce custom objects and fields for data management, lookup relationships for connecting records, a Lightning App for navigation, Record-Triggered Flows for business automation, Apex Trigger and Trigger Handler logic for stock validation, Batch Apex for pending-order processing, Scheduled Apex for automated execution, and email actions for customer reminders.

3. Data Model and Custom Objects

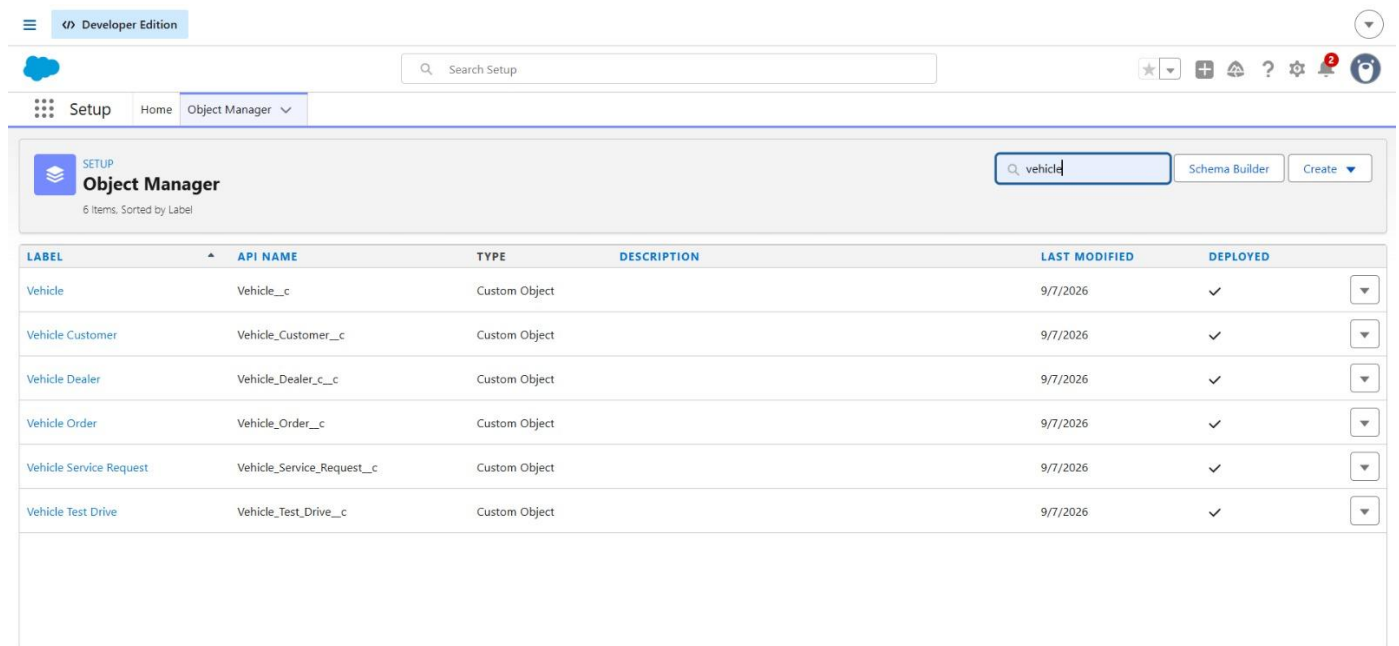
3.1 Custom Objects

The project uses custom Salesforce objects to represent the major entities involved in the vehicle management process. The main objects are Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive, and Service Request. These objects provide a structured way to store information and connect related business records.

- Vehicle – stores vehicle name, vehicle model, stock quantity, and dealer relationship.
- Vehicle Dealer – stores dealer name and dealer location information.
- Vehicle Customer – stores customer information used by order and test-drive processes.
- Vehicle Order – stores order information, selected vehicle, customer, status, and dealer assignment.
- Vehicle Test Drive – stores scheduled test-drive information and status.
- Service Request – provides a Salesforce object for managing customer service requests.

3.2 Relationships

Lookup relationships are used where one Salesforce record needs to reference another record. In particular, the Vehicle object contains a lookup relationship to the Vehicle Dealer object. The Vehicle Order and Test Drive processes also use relationships to retrieve the related customer and vehicle information required by automation.



The screenshot shows the Salesforce Object Manager interface. At the top, there's a navigation bar with 'Setup', 'Home', and 'Object Manager' tabs. Below the tabs, there's a search bar with 'vehicle' entered. The main content area displays a table of custom objects. The table has columns for LABEL, API NAME, TYPE, DESCRIPTION, LAST MODIFIED, and DEPLOYED. There are 6 items listed, all of which are Custom Objects created on 9/7/2026 and are deployed.

LABEL	API NAME	TYPE	DESCRIPTION	LAST MODIFIED	DEPLOYED
Vehicle	Vehicle__c	Custom Object		9/7/2026	✓
Vehicle Customer	Vehicle_Customer__c	Custom Object		9/7/2026	✓
Vehicle Dealer	Vehicle_Dealer_c__c	Custom Object		9/7/2026	✓
Vehicle Order	Vehicle_Order__c	Custom Object		9/7/2026	✓
Vehicle Service Request	Vehicle_Service_Request__c	Custom Object		9/7/2026	✓
Vehicle Test Drive	Vehicle_Test_Drive__c	Custom Object		9/7/2026	✓

4. Vehicle Management

4.1 Vehicle Record Management

The Vehicle object is the central record used to maintain information about vehicles available through the business. A vehicle record contains the vehicle name and the configured vehicle fields. The project specifically uses a Vehicle Model picklist and Stock

Quantity number field. These values support standardized data entry and the stock validation process.

The screenshot displays the Salesforce interface for 'WhatNext Vision Motors'. The top navigation bar includes a search bar and various tabs: Vehicles, Vehicle Test Drives, Vehicle Service Request, Vehicle Orders, Vehicle Dealers, Vehicle Customers, and a Welcome message. The main content area is titled 'Vehicle Mahindra'. On the left, the 'Details' tab is active, showing a list of fields: Vehicle Name (Mahindra), Vehicle Model (Mahindra EX), Stock Quantity (50), Price (\$5,000,000), Vehicle Dealer, Status (Available), and Created By (pavitra vanjarapu, 9/19/2026, 9:32 AM). On the right, the 'Activity' tab is active, showing a list of activities and a 'Show All Activities' button. The 'Activity' tab also includes filters for 'All time', 'All activities', and 'All types', and a 'Refresh' button.

4.2 Vehicle Model Field

The Vehicle Model field is configured as a Picklist. Using a picklist provides controlled values for the vehicle model and helps keep the stored information consistent.

4.3 Stock Quantity Field

The Stock Quantity field is configured as a Number field. It represents the quantity currently available for a vehicle. The Apex validation logic reads this value when an order is inserted or updated.

5. Dealer Management

5.1 Vehicle Dealer Object

The Vehicle Dealer object is used to maintain detailed dealer information. The object uses Dealer Name as its record name and includes Dealer Location information used by the dealer-assignment Flow.

New Vehicle Dealer

* = Required Information

Information

* Vehicle_Dealer__c Name

Dealer Name

Dealer Location

Dealer Code

Phone

Email

Owner: pavitra vanjarapu

Buttons: Cancel, Save & New, Save

5.2 Dealer Location

Dealer location is important to the order automation because the Record-Triggered Flow uses the customer's address/location and dealer location to identify the dealer record to be assigned to a pending vehicle order.

5.3 Vehicle–Dealer Association

The lookup relationship allows a Vehicle record to reference a Vehicle Dealer. This association provides the Salesforce data structure required to connect inventory information with dealer information.

6. Customer and Vehicle Order Management

6.1 Vehicle Customer Management

The Vehicle Customer object stores customer details used by the vehicle ordering and test-drive processes. The customer record is retrieved by Flow when an order or test-drive process requires customer information.

New Vehicle Customer

* = Required Information

Information

* Vehicle Customer Name

Email

Phone

Address

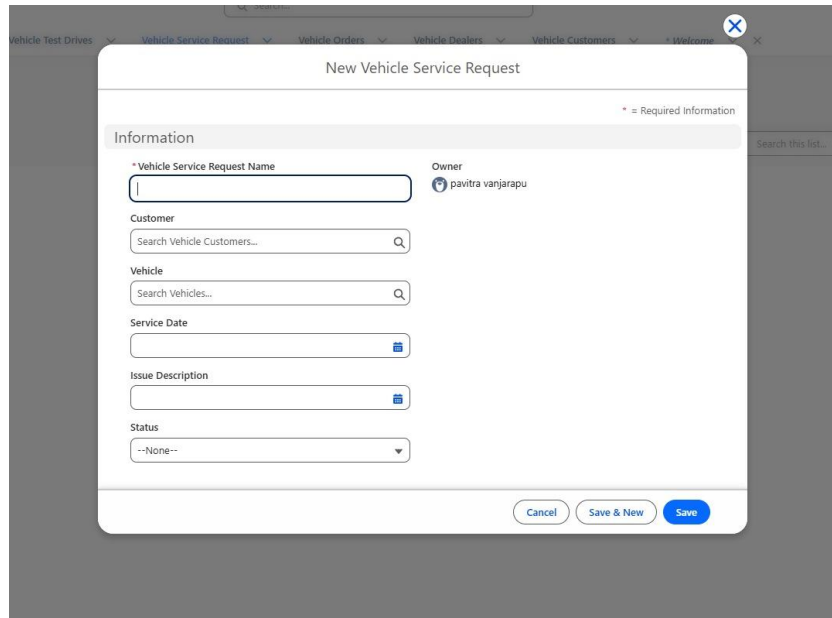
Preferred Vehicle Type: --None--

Owner: pavitra vanjarapu

Buttons: Cancel, Save & New, Save

6.2 Vehicle Order Management

The Vehicle Order object is used to record customer vehicle orders. The order contains relationships to the customer and vehicle and includes a Status field used by the automation. Pending and Confirmed statuses are important to the order-processing logic.



The screenshot shows a 'New Vehicle Service Request' form within a Salesforce application. The form is titled 'New Vehicle Service Request' and includes a close button (X) in the top right corner. A legend indicates that fields with an asterisk (*) are required information. The form is divided into sections: 'Information' (containing 'Vehicle Service Request Name' and 'Owner'), 'Customer' (with a search field 'Search Vehicle Customers...'), 'Vehicle' (with a search field 'Search Vehicles...'), 'Service Date' (with a date picker), 'Issue Description' (with a text area), and 'Status' (a dropdown menu currently showing '--None--'). At the bottom of the form are three buttons: 'Cancel', 'Save & New', and 'Save'.

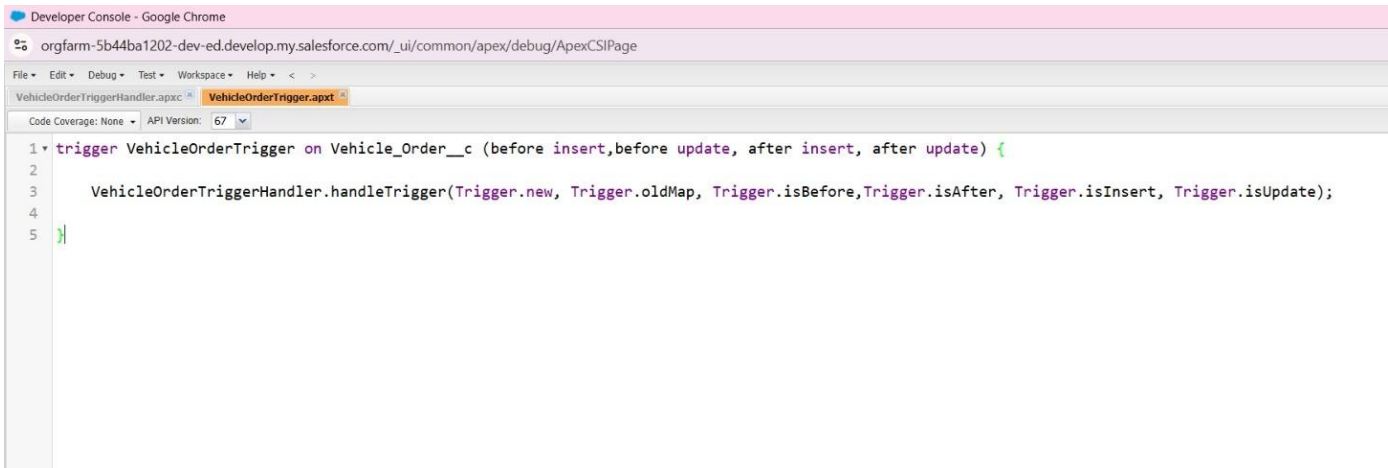
6.3 Order Lifecycle

A typical order begins as Pending when the business process requires further processing. The system checks vehicle stock before allowing the order to proceed. When stock is available and the order is confirmed, the stock update logic reduces the vehicle quantity. Pending orders can later be processed by Batch Apex when stock becomes available.

7. Apex Trigger and Trigger Handler

7.1 Trigger Architecture

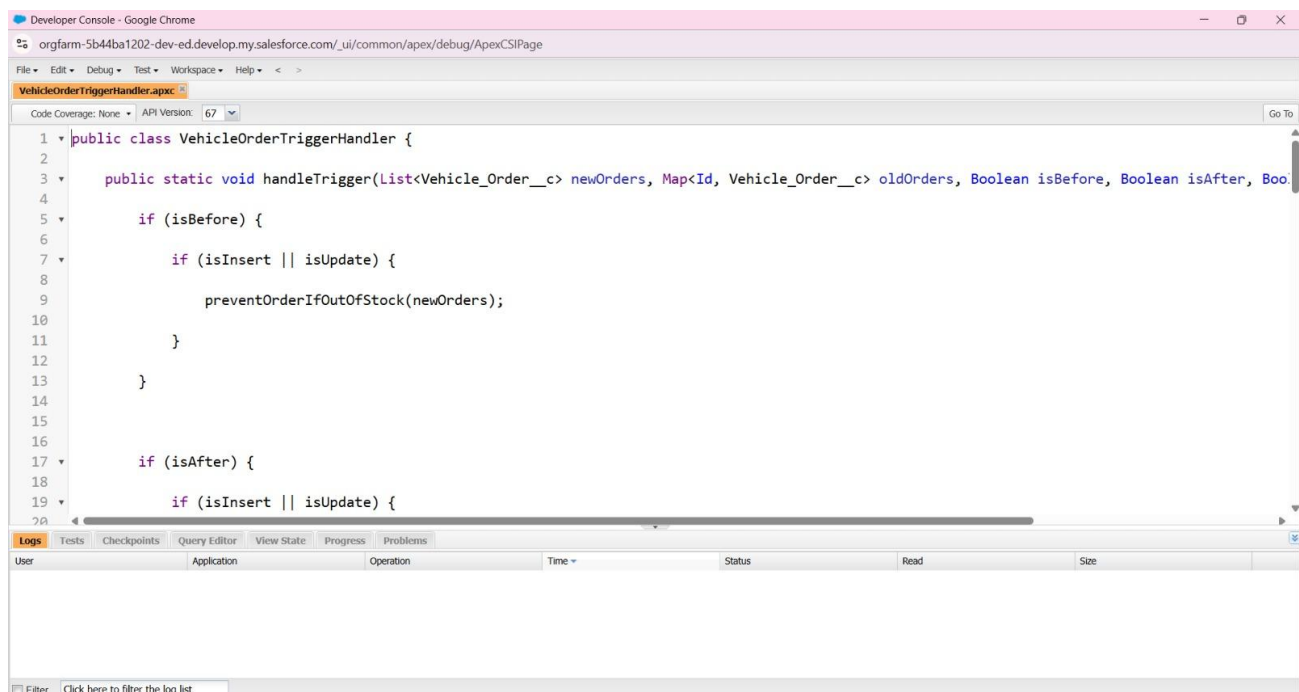
The project follows a Trigger Handler approach for Vehicle Order processing. The VehicleOrderTrigger is configured for before insert, before update, after insert, and after update events. Instead of keeping the business logic directly inside the trigger, the trigger calls VehicleOrderTriggerHandler.handleTrigger().



```
1 trigger VehicleOrderTrigger on Vehicle_Order__c (before insert,before update, after insert, after update) {  
2  
3     VehicleOrderTriggerHandler.handleTrigger(trigger.new, trigger.oldMap, trigger.isBefore, trigger.isAfter, trigger.isInsert, trigger.isUpdate);  
4  
5 }
```

7.2 Trigger Handler

The VehicleOrderTriggerHandler contains separate processing methods for validating vehicle stock and updating stock after confirmed orders. The handler receives the new records, old record map, and trigger context flags.



```
1 public class VehicleOrderTriggerHandler {  
2  
3     public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {  
4  
5         if (isBefore) {  
6  
7             if (isInsert || isUpdate) {  
8  
9                 preventOrderIfOutOfStock(newOrders);  
10  
11             }  
12  
13         }  
14  
15  
16  
17         if (isAfter) {  
18  
19             if (isInsert || isUpdate) {  
20  
21                 // Logic for updating stock after confirmed orders  
22  
23             }  
24  
25         }  
26  
27     }  
28  
29     private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> newOrders) {  
30  
31     }  
32  
33 }
```

7.3 Benefits of the Handler Structure

- Separates trigger events from business logic.
- Keeps the trigger class small and focused on event handling.
- Provides separate methods for stock validation and stock updates.
- Uses collections and bulk queries rather than placing SOQL queries inside loops.

8. Order Validation and Stock Processing

8.1 Preventing Out-of-Stock Orders

The before-trigger logic collects the Vehicle IDs associated with the incoming vehicle orders and retrieves the corresponding Vehicle records. The Stock Quantity value is then checked. If the selected vehicle has a stock quantity of zero or less, the order is rejected using addError and the user receives an out-of-stock message.

8.2 Confirmed Order Stock Update

The after-trigger processing identifies confirmed orders and collects their vehicle IDs. The corresponding vehicle records are retrieved and the stock quantity is reduced when stock is available. The updated Vehicle records are then saved.

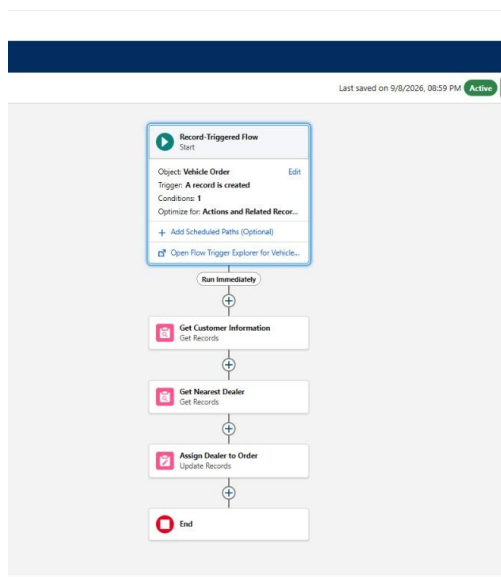
8.3 Demonstration Scenario

- Vehicle stock quantity is set to zero.
- A user attempts to create an order for that vehicle.
- The before-trigger checks the vehicle stock.
- The order is blocked with an error message.
- When stock is available, a confirmed order can be processed.
- The vehicle stock quantity is reduced after the confirmed order.

9. Automatic Dealer Assignment

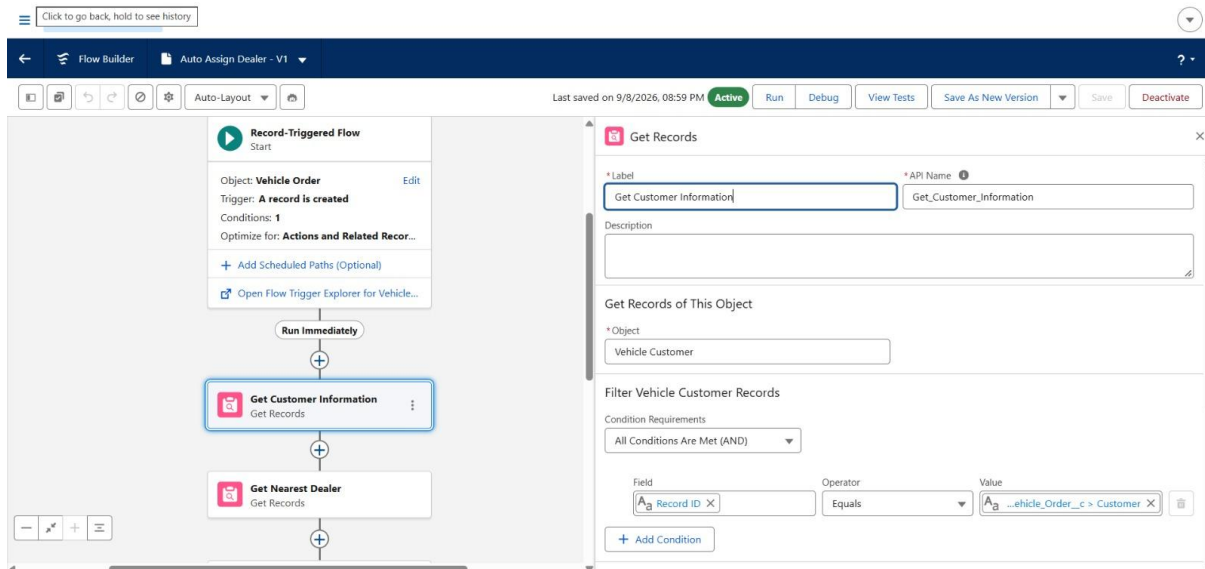
9.1 Record-Triggered Flow

A Record-Triggered Flow named Auto Assign Dealer is implemented on the Vehicle Order object. The Flow is configured to run when a Vehicle Order record is created and the Status field is Pending.



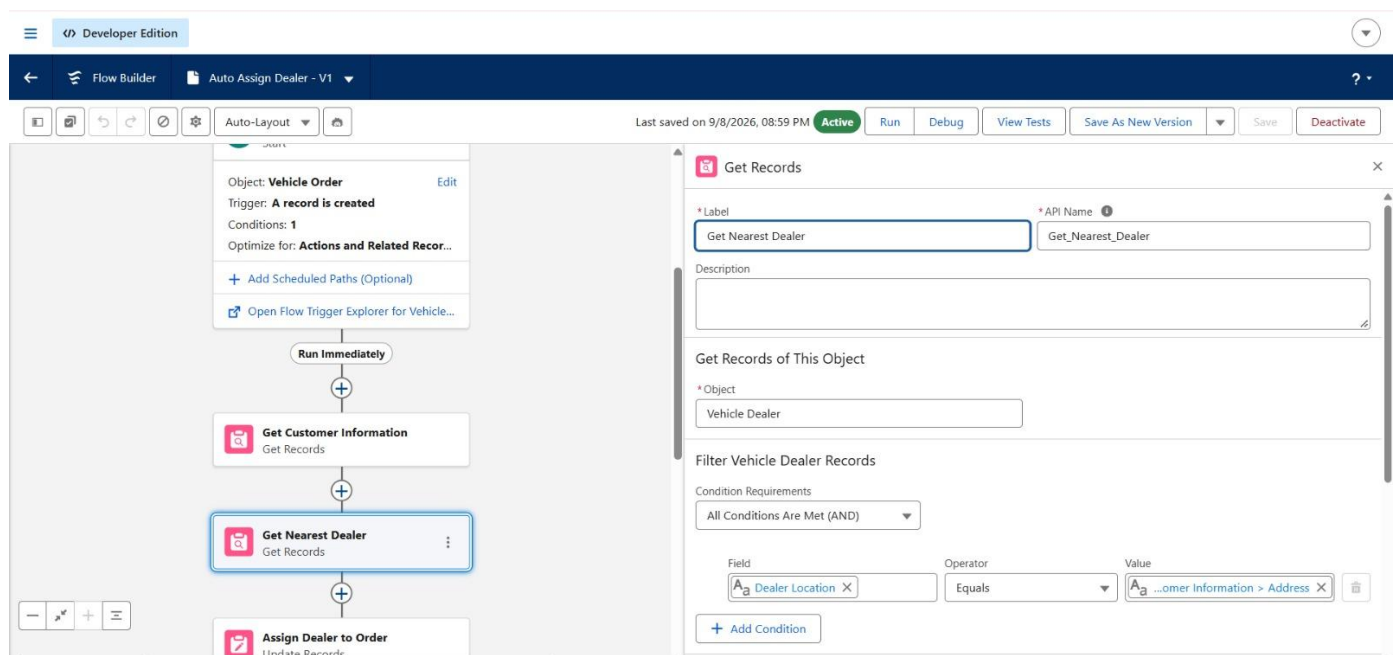
9.2 Get Customer Information

The first Get Records element retrieves the Vehicle Customer record related to the Vehicle Order. The Flow uses the customer relationship stored on the order to identify the customer record and access the customer's address/location.



9.3 Get Nearest Dealer

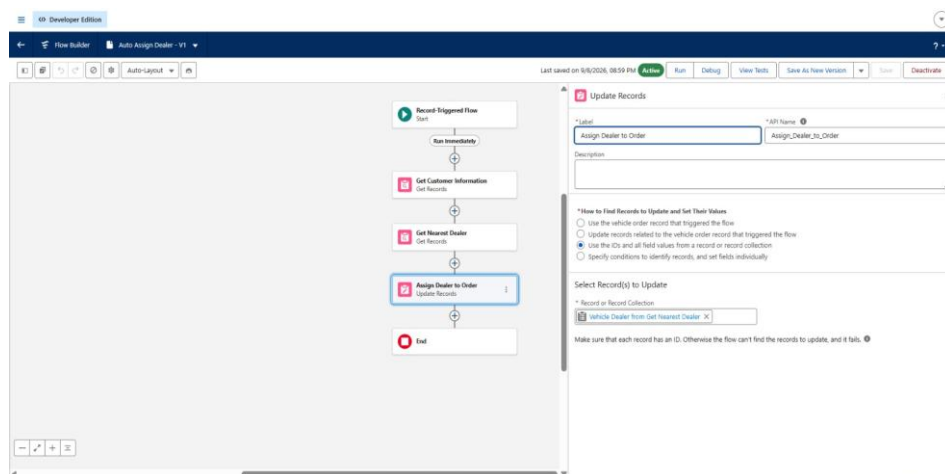
The second Get Records element retrieves the Vehicle Dealer record using the Dealer Location and the customer's address/location. The project's Flow uses this matching logic to identify the dealer record.



10. Dealer Assignment – Update and Result

10.1 Assign Dealer to Order

After the dealer record is retrieved, the Flow updates the Vehicle Order. The Update Records element is used to apply the dealer assignment to the order. The Flow is then saved and activated with the label Auto Assign Dealer.



10.2 End-to-End Dealer Assignment

The completed automation creates a simple automated sequence: a Pending Vehicle Order is created, the related customer is retrieved, the dealer matching the customer's location is retrieved, and the order is updated with the dealer. This reduces the need for manual dealer selection during order entry.

10.3 Business Benefit

Automated dealer assignment improves consistency in order processing and reduces the manual effort required to identify and enter the dealer for each customer order.

11. Test Drive Management

11.1 Vehicle Test Drive

The Vehicle Test Drive object is used to manage scheduled test drives. The test-drive record contains the information needed by the reminder automation, including the test-drive date, customer relationship, and status.

New Vehicle Test Drive

* = Required Information

Information

* **Vehicle Test Drive Name**

ramayaa

Customer

sitha

Vehicle

Mahindra

Test Drive Date

9/24/2026

Status

Scheduled,

Owner

pavitra vanjarapu

Buttons: Cancel, Save & New, Save

11.2 Scheduled Test Drive Status

The Record-Triggered Flow checks the Test Drive Status. The configured entry condition is Status = Scheduled. When a test drive is created or updated with this status, the Flow can create the scheduled reminder path.

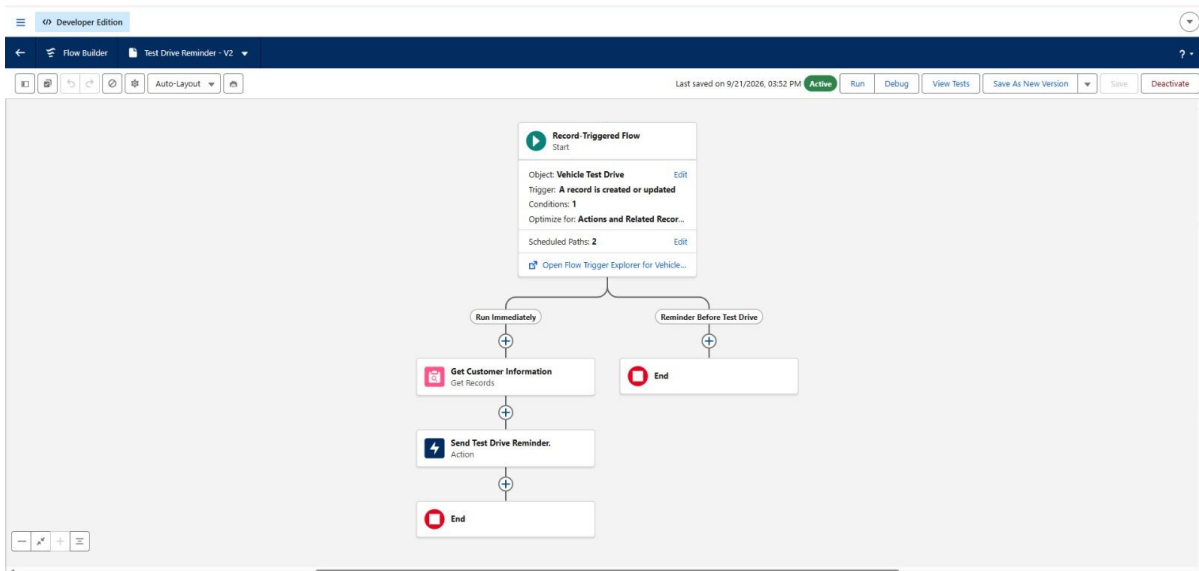
11.3 Reminder Timing

A scheduled path named Reminder Before Test Drive is configured using Test_Drive_Date__c as the time source. The offset is one day before the test-drive date

12. Automated Test Drive Reminder

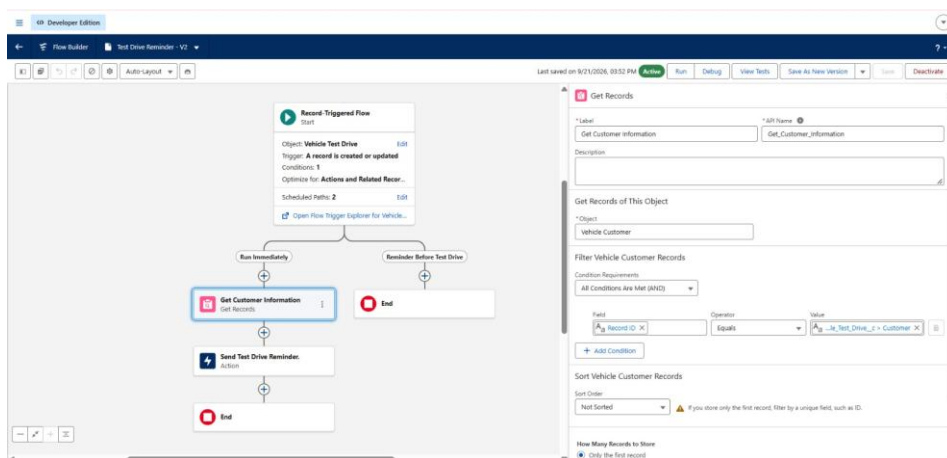
12.1 Reminder Flow

The Test Drive Reminder Flow retrieves the customer record using the Customer relationship on the Vehicle Test Drive. It then uses a Send Email action to deliver the reminder to the customer's email address.



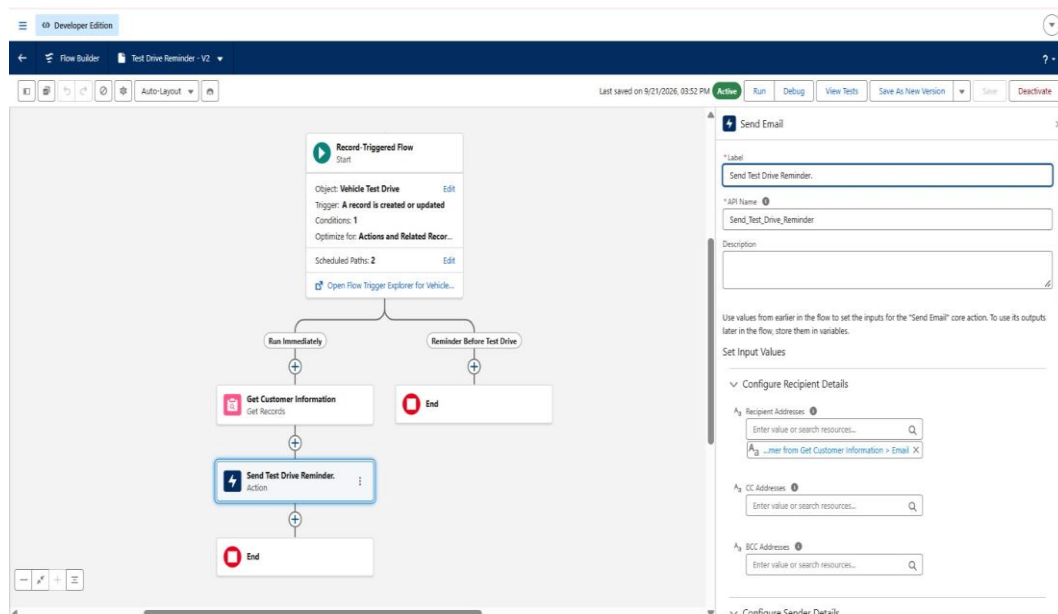
12.2 Get Customer Information

The Get Records element is configured for the Vehicle_Customer__c object and retrieves the customer whose Id matches the Customer__c value on the test-drive record. The customer email field is then available to the Send Email action.



12.3 Send Reminder Email

The Send Email action is labeled Send Test Drive Reminder. The subject is configured as "Reminder: Your Test Drive is Tomorrow!". The email is sent using the customer's email address retrieved from the related customer record.



13. Test Drive Reminder Email Result

13.1 Reminder Message

The reminder email informs the customer that the scheduled test drive is tomorrow. The message provides a simple customer-facing reminder and encourages the customer to arrive on time. The automated communication supports the project's customer experience requirement.

13.2 Email Automation Flow

- Vehicle Test Drive is created or updated.
- Status is checked for Scheduled.
- Scheduled path waits until one day before the Test Drive Date.
- Customer information is retrieved.
- Customer email address is used as the recipient.
- Send Email action sends the reminder.

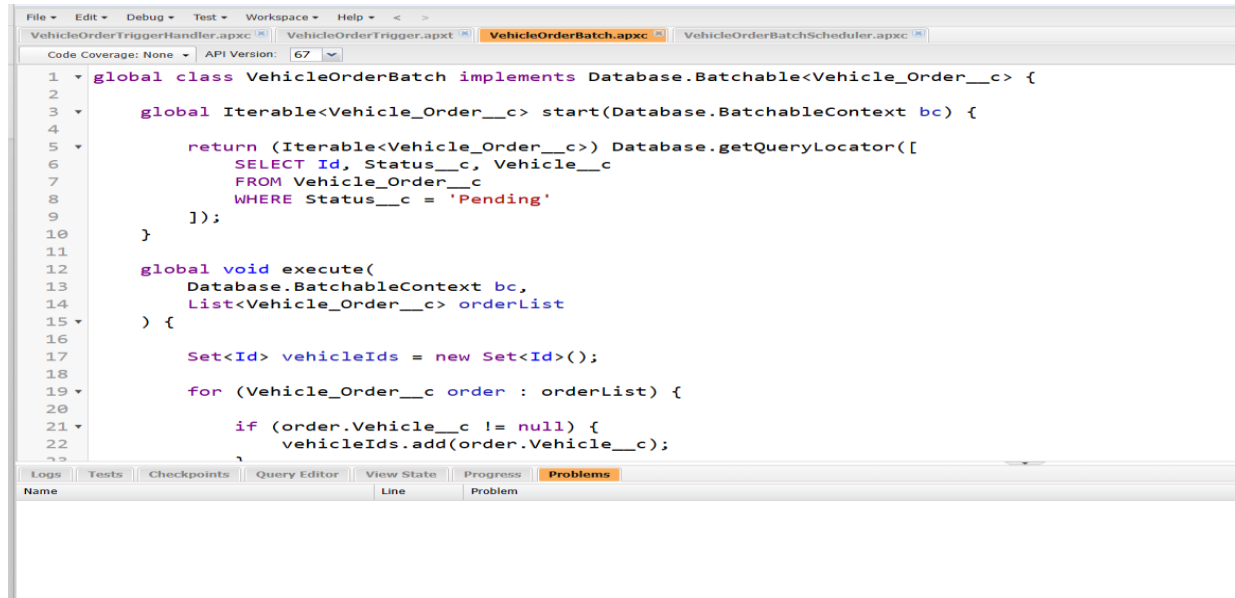
13.3 Business Benefit

The reminder automation reduces manual follow-up and provides customers with a consistent reminder before their scheduled test drive.

14. Pending Order Processing Using Batch Apex

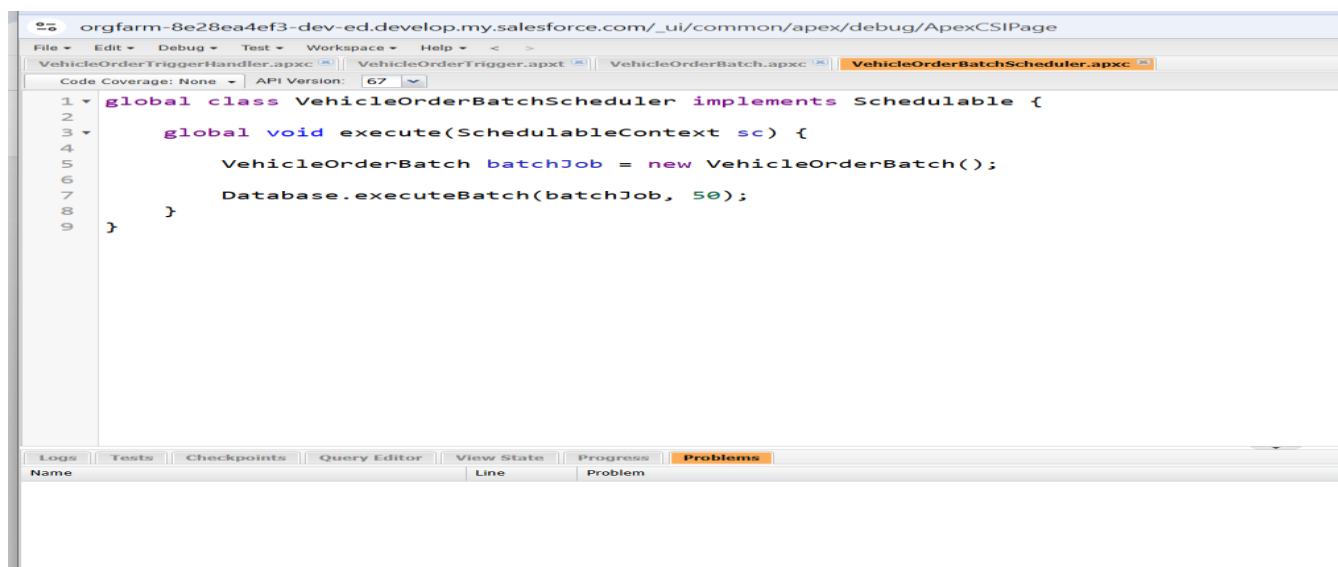
14.1 VehicleOrderBatch

The VehicleOrderBatch class implements the Database.Batchable interface. Its purpose is to process pending Vehicle Order records. The batch identifies pending orders, collects the related vehicle IDs, checks vehicle stock, and processes orders when stock is available.



14.2 Pending Order Logic

For each pending order, the batch checks whether the related vehicle exists in the retrieved vehicle-stock map. When the vehicle has available stock, the order status is changed to Confirmed and the vehicle stock quantity is reduced. The updated orders and vehicles are then saved



14.3 Batch Processing Advantage

Batch Apex provides a Salesforce mechanism for processing records in batches. This is appropriate for the project because pending orders may need to be processed periodically instead of being handled manually one record at a time.

15. Scheduled Apex and Automated Order Processing

15.1 VehicleOrderBatchScheduler

The VehicleOrderBatchScheduler class implements the Schedulable interface. Its execute method creates a VehicleOrderBatch instance and invokes Database.executeBatch with a batch size of 50.

15.2 Scheduling the Batch

The project uses System.schedule to create the scheduled job. The configured CRON expression determines the time at which the batch scheduler executes. During the demonstration, the actual configured schedule in the Salesforce org should be shown.

15.3 Scheduled Job Verification

Salesforce Setup provides a Scheduled Jobs area where the configured scheduled process can be reviewed. This allows the project implementation to be verified after scheduling.

 **Scheduled Jobs**

All Scheduled Jobs

The All Scheduled Jobs page lists all of the jobs scheduled by your users. Multiple job types may display on this page. You can delete scheduled jobs if you have the permission to do so.

You have currently used 1 scheduled Apex jobs out of an allowed organization limit of 100 active or scheduled jobs. To learn about how this limit is calculated and what contributes to it see the [Lightning Platform Apex Limits](#) topic.

View: All Scheduled Jobs [Create New View](#)

[Schedule Apex](#)

Action	Job Name +	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Daily Vehicle Order Processing	vanjarapu_pavitra	9/8/2026, 8:51 AM	9/20/2026, 12:04 PM	9/21/2026, 12:00 PM	Scheduled Apex	08ebm0000v4BvL
Del	Metalytics Data Loader Job for Org : 00Dbm00000xcgvi	User Integration	9/2/2026, 5:07 AM	9/20/2026, 12:39 PM	9/21/2026, 12:39 PM	Autonomous Data Loader Job	08ebm00000u6iTr
	Program Milestone Computation Cron Job	Process, Automated	9/2/2026, 5:07 AM	9/21/2026, 12:00 AM	9/21/2026, 6:59 AM	Program Milestone Computation Cron Job	08ebm00000u6iTp
	Program Status Update Cron Job	Process, Automated	9/2/2026, 5:07 AM	9/20/2026, 8:01 PM	9/21/2026, 5:00 AM	Program Status Update Cron Job	08ebm00000u6iTq

A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | Other | **All**

16. End-to-End Project Workflow

16.1 Overall Process

The completed solution connects several Salesforce components into one vehicle-management workflow. Vehicle records maintain stock, customer records provide customer information, and Vehicle Orders connect customers with vehicles. Flows automate dealer assignment and test-drive reminders, while Apex validates stock and updates inventory. Batch Apex and Scheduled Apex provide automated processing for pending orders.

16.2 End-to-End Scenario

- A vehicle and its available stock are maintained in Salesforce.
- A customer record is created and used for order or test-drive processing.
- A vehicle order is created with the required status.
- Apex validates the vehicle's stock before the order is accepted.
- A Pending order can be automatically associated with the appropriate dealer.
- A Confirmed order reduces the vehicle stock quantity.
- A scheduled test drive triggers a reminder one day before the test-drive date.
- Pending orders can later be processed by Batch Apex when stock becomes available.
- Scheduled Apex automatically invokes the batch process according to the configured schedule.

16.3 Project Outcome

The project demonstrates how Salesforce configuration, Flow automation, and Apex programming can be combined to implement practical automotive business requirements. The solution centralizes information and automates important order and customer processes while reducing repetitive manual activities.

17. Testing and Demonstration Evidence

17.1 Functional Test Scenarios

The following scenarios can be demonstrated in the Salesforce org to provide evidence that the main automation features are working as intended.

- Out-of-stock vehicle: set Stock Quantity to 0 and attempt to create an order; verify that the Apex validation prevents the order.
- Available vehicle: create a confirmed order for a vehicle with available stock; verify that the stock quantity decreases.

- Dealer assignment: create a Pending order with a customer location matching a dealer location; verify the dealer assignment.
- Test-drive reminder: create a scheduled test drive and verify the scheduled reminder Flow configuration and email action.
- Pending order processing: create a Pending order, add vehicle stock, and execute/verify the Batch Apex processing.
- Scheduled processing: verify the VehicleOrderBatchScheduler in Salesforce Scheduled Jobs

18. Conclusion

The WhatNext Vision Motors Salesforce application successfully integrates Salesforce custom objects, fields, relationships, Lightning App configuration, Record-Triggered Flows, Apex Trigger and Trigger Handler logic, Batch Apex, Scheduled Apex, and automated email communication. The system provides a centralized platform for managing vehicles, dealers, customers, orders, test drives, and service requests.

The implementation addresses the project's major business requirements. Vehicle stock is checked before orders are accepted, confirmed orders update stock, dealer assignment is automated using customer location, test-drive reminders are scheduled automatically, and pending orders can be processed when stock becomes available. Together, these features demonstrate the practical use of Salesforce automation and Apex programming for a real-world vehicle management scenario.

19. Technology Summary

- Platform: Salesforce
- Application: WhatNext Vision Motors Lightning App
- Data Management: Custom Objects, Custom Fields, Lookup Relationship
- Automation: Record-Triggered Flow and Scheduled Path
- Email: Send Email action for test-drive reminders
- Apex: VehicleOrderTrigger and VehicleOrderTriggerHandler
- Batch Processing: VehicleOrderBatch
- Scheduling: VehicleOrderBatchScheduler
- Main Objects: Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive, Service Request

20. Project Demonstration Checklist

- Open the WhatNext Vision Motors Lightning application.
- Show Vehicle and Vehicle Dealer records.
- Show the Vehicle–Dealer lookup relationship.
- Demonstrate out-of-stock order validation.
- Demonstrate confirmed order stock reduction.
- Show the Auto Assign Dealer Flow and its result.
- Show the Test Drive Reminder Flow.
- Show the generated test-drive reminder email.
- Show VehicleOrderTrigger and VehicleOrderTriggerHandler.
- Show VehicleOrderBatch and VehicleOrderBatchScheduler.